Write a program to identify and count the total number of objects present on a web page including links, images, buttons, and input elements. Display categorized counts.

- Click File → New → Maven Project.
- Enter the project name `WebPageObjectCount`.
- Open `pom.xml` and add the following dependencies inside the `<dependencies>` tag.

Replace the existing `pom.xml` content with this code.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.selenium</groupId>
    <artifactId>WebPageObjectCount</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
```

```
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.35.0</version>
      </dependency>
    </dependencies>

</project>
```

Save the file. Right-click the project and select Maven → Update Project if necessary.

Step 3: Create the Java class

1. Expand `src/main/java`.
2. Right-click it and select New → Package.
3. Enter package name `com.selenium`.
4. Right-click the package and select New → Class.
5. Enter class name `WebPageObjectCount`.
6. Copy the complete code below.

Step 4: Complete Java program

```
package com.selenium;


import java.time.Duration;
import java.util.List;


import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;


public class WebPageObjectCount {
```

```
public static void main(String[] args) {

    WebDriver driver = null;

    try {
        // Launch Chrome browser
        driver = new ChromeDriver();

        // Maximize browser window
        driver.manage().window().maximize();

        // Set implicit wait
        driver.manage().timeouts()
              .implicitlyWait(Duration.ofSeconds(5));

        // Open website
        driver.get("https://www.selenium.dev/");

        // Wait for the page to load
        Thread.sleep(3000);

        // Count links
        List<WebElement> links =
               driver.findElements(By.tagName("a"));
        int linkCount = links.size();
```

```
// Count images
List<WebElement> images =
      driver.findElements(By.tagName("img"));
int imageCount = images.size();

// Count button elements
List<WebElement> buttons =
      driver.findElements(By.tagName("button"));
int buttonCount = buttons.size();

// Count input elements
List<WebElement> inputs =
      driver.findElements(By.tagName("input"));
int inputCount = inputs.size();

// Calculate total objects
int totalObjects = linkCount + imageCount
            + buttonCount + inputCount;

// Display categorized counts
System.out.println("================================");
System.out.println(" WEB PAGE OBJECT COUNT REPORT");
System.out.println("================================");

System.out.println("Website URL: " + driver.getCurrentUrl());
System.out.println("Page Title: " + driver.getTitle());
```

```
            System.out.println("---------------------------------");
            System.out.println("Total Links          : " + linkCount);
            System.out.println("Total Images         : " + imageCount);
            System.out.println("Total Buttons        : " + buttonCount);
            System.out.println("Total Input Elements : " + inputCount);
            System.out.println("---------------------------------");
            System.out.println("TOTAL OBJECTS        : " + totalObjects);
            System.out.println("=================================");

        } catch (Exception e) {
            System.out.println("Error occurred: " + e.getMessage());
            e.printStackTrace();

        } finally {
            // Keep browser open briefly, then close
            if (driver != null) {
                try {
                    Thread.sleep(3000);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

                driver.quit();
                System.out.println("Browser closed successfully.");
            }
        }
    }
```

}

Step 5: Run the program

1. Save the Java file using `Ctrl + S`.
2. Right-click `WebPageObjectCount.java`.
3. Select Run As → Java Application.
4. Chrome will open the Selenium website.
5. The categorized counts will appear in the Eclipse/STS Console.
6. After displaying the output, the browser will close automatically.

Output

```
WEB PAGE OBJECT COUNT REPORT
==================================
Website URL: https://www.selenium.dev/
Page Title: Selenium

----------------------------------
Total Links          : 25
Total Images         : 8
Total Buttons        : 3
Total Input Elements : 1
----------------------------------
TOTAL OBJECTS        : 37
==================================
Browser closed successfully.
```

Implement screenshot capture on test failure and integrate logging mechanism using Log4j. Generate execution logs and analyze error messages.

Step 1: Create a Maven project

1. Open Eclipse or STS.
2. Select File → New → Maven Project.
3. Enter the project name `ScreenshotLog4j`.
4. Under `src/test/java`, create the package `com.selenium.test`.
5. Create a Java class named `ScreenshotLog4jTest`.

Step 2: Configure `pom.xml`

Replace your existing `pom.xml` with this complete code. It includes Selenium, TestNG, Log4j 2, and the Maven Surefire plugin to execute TestNG tests.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"

        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">


   <modelVersion>4.0.0</modelVersion>


   <groupId>com.selenium</groupId>

   <artifactId>ScreenshotLog4j</artifactId>

   <version>1.0-SNAPSHOT</version>


   <properties>

      <maven.compiler.source>17</maven.compiler.source>

      <maven.compiler.target>17</maven.compiler.target>

      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
</properties>

<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.35.0</version>
  </dependency>

  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.11.0</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>2.25.2</version>
  </dependency>

  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.25.2</version>
  </dependency>
```

```
  </dependencies>


  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>testng.xml</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>


</project>
```

Save the file. Right-click the project and choose Maven → Update Project.

Step 3: Create the Log4j configuration file

1. Right-click the project.
2. Select New → Folder.
3. Name the folder `src/test/resources` if it does not already exist.
4. Create a file named `log4j2.xml` inside that folder.
5. Paste this configuration.

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
    <Appenders>
        <Console name="Console" target="SYSTEM_OUT">
```

```
            <PatternLayout
                pattern="%d{HH:mm:ss} %-5level %c{1} - %msg%n"/>
        </Console>

        <File name="FileLogger"
              fileName="logs/execution.log"
              append="false">
            <PatternLayout
                pattern="%d{yyyy-MM-dd HH:mm:ss} %-5level %c{1} - %msg%n"/>
        </File>
    </Appenders>

    <Loggers>
        <Root level="info">
            <AppenderRef ref="Console"/>
            <AppenderRef ref="FileLogger"/>
        </Root>
    </Loggers>
</Configuration>
```

This configuration sends logs to both the Console and `logs/execution.log`. The program will create the `logs` folder before the tests run.

Step 4: Create the TestNG listener

Create another Java class in `com.selenium.test` named `ScreenshotListener`.

This listener automatically captures a screenshot whenever a TestNG test fails.

```
package com.selenium.test;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.testng.ITestListener;
import org.testng.ITestResult;

public class ScreenshotListener implements ITestListener {

    private static final Logger log =
            LogManager.getLogger(ScreenshotListener.class);

    @Override
    public void onTestFailure(ITestResult result) {

        Object testObject = result.getInstance();
```

```
        if (!(testObject instanceof ScreenshotLog4jTest)) {
            log.error("Unable to access WebDriver for screenshot.");
            return;
        }

        ScreenshotLog4jTest test =
                (ScreenshotLog4jTest) testObject;

        WebDriver driver = test.getDriver();

        if (driver == null) {
            log.error("Screenshot not captured: WebDriver is null.");
            return;
        }

        try {
            Path screenshotDir = Paths.get("screenshots")
                                      .toAbsolutePath();

            Files.createDirectories(screenshotDir);

            String timestamp = LocalDateTime.now().format(
                    DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss_SSS"));

            String testName = result.getMethod().getMethodName();

            File source = ((TakesScreenshot) driver)
                    .getScreenshotAs(OutputType.FILE);

            Path destination = screenshotDir.resolve(
                    testName + "_" + timestamp + ".png");

            Files.copy(source, destination,
                    StandardCopyOption.REPLACE_EXISTING);

            log.error("TEST FAILED: {}", testName);
            log.error("Screenshot saved at: {}", destination);
            log.error("Failure reason: ",
                    result.getThrowable());

        } catch (IOException | RuntimeException e) {
            log.error("Could not save failure screenshot.", e);
        }
    }
}
```

Step 5: Create the main test class

Open `ScreenshotLog4jTest.java` and paste this complete code.

The second test deliberately uses an incorrect expected title so you can verify that screenshot capture and failure logging work. After confirming the practical, replace `"Incorrect Title"` with `"Selenium"` to make the test pass.

```
package com.selenium.test;
```

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.ITestResult;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;

@Listeners(ScreenshotListener.class)
public class ScreenshotLog4jTest {

    private static final Logger log =
            LogManager.getLogger(ScreenshotLog4jTest.class);

    private WebDriver driver;

    public WebDriver getDriver() {
        return driver;
    }

    @BeforeMethod
    public void setUp() throws IOException {

        Files.createDirectories(Paths.get("logs"));
        Files.createDirectories(Paths.get("screenshots"));

        log.info("Starting Chrome browser.");

        driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.manage().timeouts()
              .implicitlyWait(Duration.ofSeconds(5));

        driver.get("https://www.selenium.dev/");

        log.info("Website opened successfully.");
        log.info("Page title: {}", driver.getTitle());
    }

    @Test(priority = 1)
    public void verifyPageTitle() {

        log.info("Executing verifyPageTitle test.");

        String actualTitle = driver.getTitle();

        Assert.assertTrue(
```

```
            actualTitle.contains("Selenium"),
            "Page title does not contain Selenium.");

    log.info("Page title validation passed.");
}

@Test(priority = 2)
public void demonstrateFailureScreenshot() {

    log.info("Executing failure demonstration test.");

    String actualTitle = driver.getTitle();

    log.info("Actual page title: {}", actualTitle);

    // Intentionally incorrect to demonstrate failure handling
    Assert.assertEquals(
            actualTitle,
            "Incorrect Title",
            "Expected title does not match actual title.");
}

@AfterMethod(alwaysRun = true)
public void tearDown(ITestResult result) {

    try {
        // Capture screenshot before closing the browser
        if (result.getStatus() == ITestResult.FAILURE
                && driver != null) {

            try {
                Path screenshotDir = Paths.get("screenshots")
                                          .toAbsolutePath();

                Files.createDirectories(screenshotDir);

                String timestamp = LocalDateTime.now().format(
                        DateTimeFormatter.ofPattern(
                                "yyyyMMdd_HHmmss_SSS"));

                String screenshotName =
                        result.getMethod().getMethodName()
                        + "_afterFailure_" + timestamp + ".png";

                Path destination =
                        screenshotDir.resolve(screenshotName);

                Path source = Paths.get(
                        ((TakesScreenshot) driver)
                        .getScreenshotAs(OutputType.FILE)
                        .getAbsolutePath());

                Files.copy(source, destination);

                log.error("Failure screenshot saved: {}",
                          destination);

            } catch (Exception e) {
                log.error("Failed to capture screenshot.", e);
            }
```

```
                log.error("Test failed: {}",
                          result.getMethod().getMethodName());

                if (result.getThrowable() != null) {
                    log.error("Failure details:",
                              result.getThrowable());
                }

            } else if (result.getStatus() == ITestResult.SUCCESS) {
                log.info("Test passed: {}",
                         result.getMethod().getMethodName());
            }

        } finally {
            if (driver != null) {
                log.info("Closing browser.");
                driver.quit();
                driver = null;
            }
        }
    }
}
```

Why capture the screenshot in `@AfterMethod` as well? It captures the browser state before `driver.quit()`, even if the TestNG listener runs after the browser has closed. The listener also records the failure and attempts screenshot capture.

Step 6: Create `testng.xml`

Create a file named `testng.xml` in the project root, at the same level as `pom.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Screenshot and Logging Suite">
    <test name="Selenium Failure Screenshot Test">
        <classes>
            <class name="com.selenium.test.ScreenshotLog4jTest"/>
        </classes>
    </test>
</suite>
```

Step 7: Execute the test

Option A — Run using TestNG

1. Right-click `testng.xml`.
2. Select Run As → TestNG Suite.
3. Chrome opens the Selenium website.
4. The first test validates the page title.
5. The second test deliberately fails because the expected title is incorrect.

6. The failure screenshot is captured before the browser closes.

Develop a script to automate file upload and download functionality. Validate successful upload and verify file existence after download.

Step 1: Create a Maven project

1. Open Eclipse or STS.
2. Select File → New → Maven Project.
3. Enter the project name `FileUploadDownload`.
4. Create a package named `com.selenium`.
5. Create a Java class named `FileUploadDownloadTest`.

Step 2: Configure `pom.xml`

Replace the existing `pom.xml` content with the following code.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"

        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">


   <modelVersion>4.0.0</modelVersion>


   <groupId>com.selenium</groupId>

   <artifactId>FileUploadDownload</artifactId>

   <version>1.0-SNAPSHOT</version>


   <properties>

      <maven.compiler.source>17</maven.compiler.source>

      <maven.compiler.target>17</maven.compiler.target>

      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
  </properties>


  <dependencies>
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.35.0</version>
    </dependency>
  </dependencies>


</project>
```

Save the file. If required, right-click the project and select Maven → Update Project.

Step 3: Complete Java program

This program uses the DemoQA Upload and Download page:

`https://demoqa.com/upload-download`

It creates a test file automatically, uploads it, downloads a sample file, and verifies both operations.

```
package com.selenium;


import java.nio.file.Files;

import java.nio.file.Path;

import java.nio.file.Paths;

import java.time.Duration;

import java.util.HashMap;

import java.util.Map;


import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class FileUploadDownloadTest {

    public static void main(String[] args) {

        WebDriver driver = null;

        try {
            // Create testdata and downloads folders
            Path uploadFolder = Paths.get("testdata")
                                     .toAbsolutePath();
            Path downloadFolder = Paths.get("downloads")
                                       .toAbsolutePath();

            Files.createDirectories(uploadFolder);
            Files.createDirectories(downloadFolder);

            // Create a test file for uploading
            Path uploadFile = uploadFolder.resolve("upload.txt");
            Files.writeString(
                  uploadFile,
                  "This is a Selenium file upload test.");
```

```
// Configure Chrome download location
Map<String, Object> preferences = new HashMap<>();
preferences.put("download.default_directory",
            downloadFolder.toString());
preferences.put("download.prompt_for_download", false);
preferences.put("download.directory_upgrade", true);

ChromeOptions options = new ChromeOptions();
options.setExperimentalOption("prefs", preferences);

// Start Chrome
driver = new ChromeDriver(options);
driver.manage().window().maximize();
driver.manage().timeouts()
    .implicitlyWait(Duration.ofSeconds(5));

WebDriverWait wait =
      new WebDriverWait(driver, Duration.ofSeconds(20));

// Open upload/download page
driver.get("https://demoqa.com/upload-download");

System.out.println("===== FILE UPLOAD TEST =====");

// Upload the file using the file input
WebElement fileInput =
```

```
        driver.findElement(By.id("uploadFile"));

fileInput.sendKeys(uploadFile.toString());

// Validate upload confirmation
WebElement uploadResult = wait.until(
        d -> d.findElement(By.id("uploadedFilePath")));

String resultText = uploadResult.getText();

System.out.println("Upload result: " + resultText);

if (resultText.contains("upload.txt")) {
    System.out.println("File upload validation: PASS");
} else {
    System.out.println("File upload validation: FAIL");
}

System.out.println("\n===== FILE DOWNLOAD TEST =====");

// Remove an old copy so the test checks a new download
Path downloadedFile =
        downloadFolder.resolve("sampleFile.jpeg");

Files.deleteIfExists(downloadedFile);

// Click the Download button
```

```
driver.findElement(By.id("downloadButton")).click();

// Wait until the download finishes
long endTime = System.currentTimeMillis() + 30000;

while (System.currentTimeMillis() < endTime) {
   boolean fileExists = Files.exists(downloadedFile);

   boolean downloadInProgress;
   try (var files = Files.list(downloadFolder)) {
      downloadInProgress = files.anyMatch(
            p -> p.getFileName().toString()
                .endsWith(".crdownload"));
   }

   if (fileExists && !downloadInProgress) {
      break;
   }

   Thread.sleep(500);
}

// Verify downloaded file
if (Files.exists(downloadedFile)
      && Files.size(downloadedFile) > 0) {

   System.out.println("Downloaded file: "
```

```
                    + downloadedFile);
            System.out.println("File exists: YES");
            System.out.println("File size: "
                    + Files.size(downloadedFile)
                    + " bytes");
            System.out.println("File download validation: PASS");

        } else {
            System.out.println("File download validation: FAIL");
            System.out.println(
                "File was not downloaded within 30 seconds.");
        }

        System.out.println("\n===== TEST EXECUTION COMPLETED =====");

        Thread.sleep(3000);

    } catch (Exception e) {
        System.out.println("Test execution error: "
                + e.getMessage());
        e.printStackTrace();

    } finally {
        if (driver != null) {
            driver.quit();
            System.out.println("Browser closed.");
        }
```

```
    }
  }
}
```

Step 4: Run the program

1. Save the Java file using `Ctrl + S`.
2. Right-click `FileUploadDownloadTest.java`.
3. Select Run As → Java Application.
4. Chrome opens the DemoQA page.
5. The program uploads `upload.txt` and checks the upload confirmation.
6. It downloads `sampleFile.jpeg` and checks whether the file exists.
7. View the results in the Eclipse/STS Console.

Step 5: Verify the files

After execution, your project directory should contain these folders and files:

FileUploadDownload

testdata

upload.txt — file created for uploading

Upload

downloads

sampleFile.jpeg — downloaded file

output

```
===== FILE UPLOAD TEST =====
Upload result: C:\fakepath\upload.txt
File upload validation: PASS

===== FILE DOWNLOAD TEST =====
Downloaded file: ...\downloads\sampleFile.jpeg
File exists: YES
File size: [actual size] bytes
File download validation: PASS

===== TEST EXECUTION COMPLETED =====
Browser closed.
`
```

Create and execute automated test cases using TestNG annotations. Group test cases, define priorities, and generate HTML test reports.

Step 1: Create a Maven project

1. Open Eclipse or STS.
2. Select File → New → Maven Project.
3. Enter the project name `TestNGAutomation`.
4. Under `src/test/java`, create a package named `com.selenium.test`.
5. Create a Java class named `TestNGAutomationTest`.

Step 2: Configure `pom.xml`

Replace the existing content with the following complete code.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.selenium</groupId>
    <artifactId>TestNGAutomation</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.35.0</version>
        </dependency>

        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
```

```
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.4</version>
                <configuration>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>
                    </suiteXmlFiles>
                </configuration>
            </plugin>
        </plugins>
    </build>

</project>
```

Save the file. Right-click the project and select Maven → Update Project.

Step 3: Create the Java test class

Create `TestNGAutomationTest.java` inside `src/test/java/com/selenium/test`.

Copy the complete code below.

```
package com.selenium.test;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class TestNGAutomationTest {

    WebDriver driver;

    @BeforeMethod
    public void setUp() {
        System.out.println("\nStarting browser...");

        driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.manage().timeouts()
              .implicitlyWait(Duration.ofSeconds(5));

        driver.get("https://www.selenium.dev/");

        System.out.println("Website opened successfully.");
    }

    @Test(priority = 1, groups = {"smoke", "regression"})
    public void verifyPageTitle() {
        System.out.println("Test 1: Verify page title");
```

```
    String actualTitle = driver.getTitle();

    System.out.println("Page title: " + actualTitle);

    Assert.assertTrue(
            actualTitle.contains("Selenium"),
            "Page title validation failed.");

    System.out.println("Page title test PASSED.");
}

@Test(priority = 2, groups = {"smoke"})
public void verifyCurrentURL() {
    System.out.println("Test 2: Verify website URL");

    String actualURL = driver.getCurrentUrl();

    System.out.println("Current URL: " + actualURL);

    Assert.assertTrue(
            actualURL.contains("selenium.dev"),
            "URL validation failed.");

    System.out.println("URL test PASSED.");
}

@Test(priority = 3, groups = {"regression"})
public void verifyHeading() {
    System.out.println("Test 3: Verify page heading");

    WebElement heading =
            driver.findElement(By.tagName("h1"));

    Assert.assertTrue(
            heading.isDisplayed(),
            "Page heading is not displayed.");

    System.out.println("Heading: " + heading.getText());
    System.out.println("Heading test PASSED.");
}

@Test(priority = 4, groups = {"regression"})
public void verifyLinksPresent() {
    System.out.println("Test 4: Verify links");

    int linkCount =
            driver.findElements(By.tagName("a")).size();

    System.out.println("Total links: " + linkCount);

    Assert.assertTrue(
            linkCount > 0,
            "No links found on the page.");

    System.out.println("Links test PASSED.");
}

@AfterMethod(alwaysRun = true)
public void tearDown() {
    if (driver != null) {
```

```
            driver.quit();
            driver = null;
            System.out.println("Browser closed.");
        }
    }
}
```

Step 4: Create `testng.xml`

Create a file named `testng.xml` in the project root, at the same level as `pom.xml`.

This configuration executes all four tests and enables both groups.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Selenium Test Suite">

    <test name="Smoke and Regression Tests">

        <groups>
            <run>
                <include name="smoke"/>
                <include name="regression"/>
            </run>
        </groups>

        <classes>
            <class name="com.selenium.test.TestNGAutomationTest"/>
        </classes>

    </test>

</suite>
```

Important: Keep `testng.xml` outside `src/test/java`, directly in the project root. Ensure the XML declaration is the first line.

Step 5: Understand TestNG annotations

Annotation	Purpose
`@BeforeMethod`	Runs before each test method
`@Test`	Marks a method as a test case
`@AfterMethod`	Runs after each test method
`priority`	Controls test execution order; lower numbers run first
`groups`	Organizes test cases into categories

`Assert.assertTrue()`	Validates whether a condition is true

Step 6: Execute the test cases

Option A: Run through Eclipse or STS

1. Save all files.
2. Right-click `testng.xml`.
3. Select Run As → TestNG Suite.
4. Chrome opens and runs the test cases one by one.
5. The browser closes after each test.
6. Open the Console to see the test results.

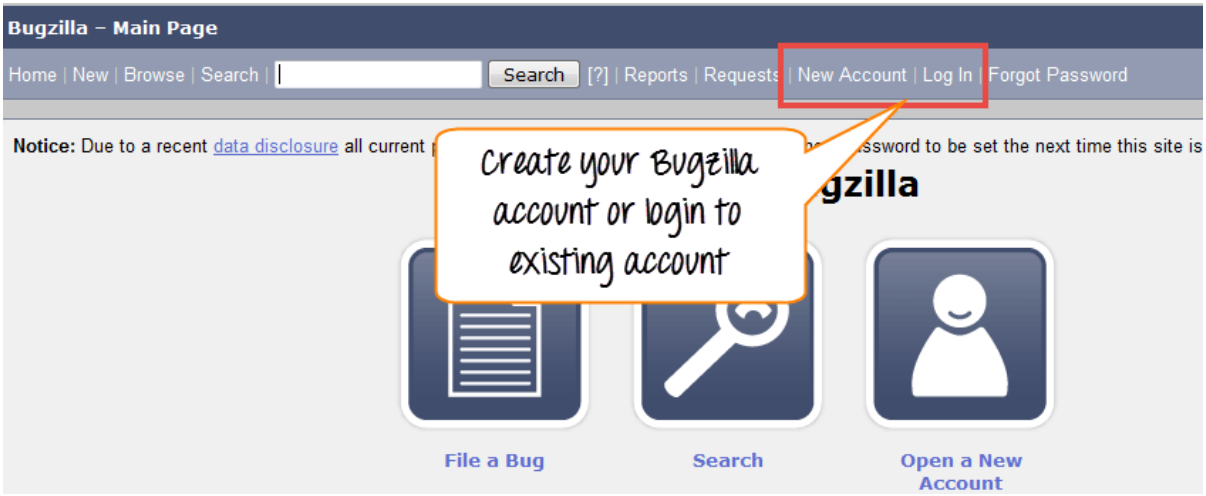
Bugzilla is an open-source issue/bug tracking system that allows developers to keep track of outstanding problems with their product.

Bugzilla is a Defect tracking tool, however, it can be used as a test management tool as such it can be easily linked with other Test Case management tools like Quality Center, Testlink etc.

Key features of Bugzilla includes

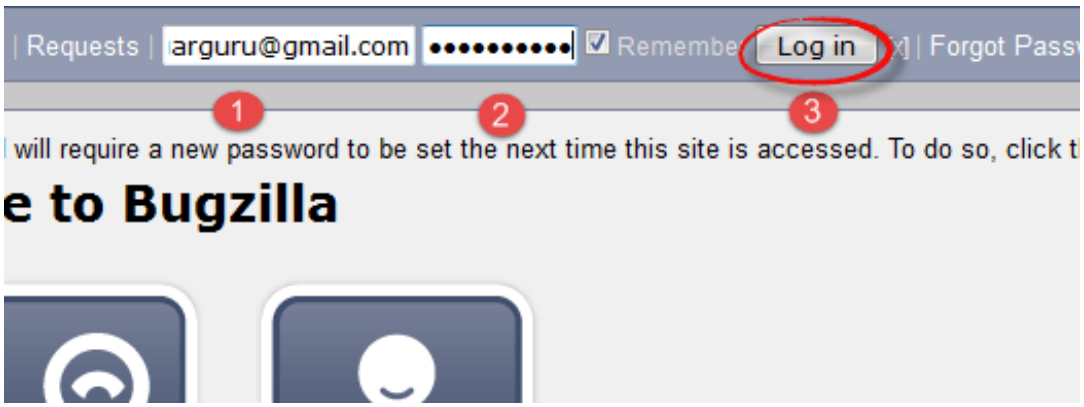
- Advanced search capabilities
- E-mail Notifications
- Modify/file Bugs by e-mail
- Time tracking
- Strong security
- Customization
- Localization

Step 1) Use the following link for your hands-on. To create an account in Bugzilla tool or to login into the existing account go to **New Account or Log in** option in the main menu.

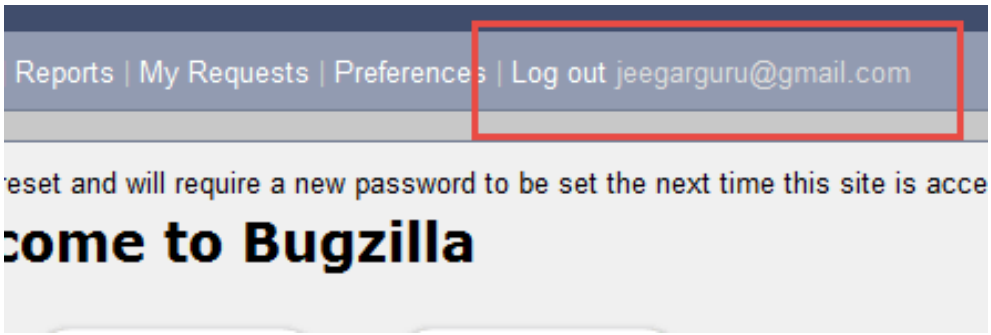


Step 2) Now, enter your personal details to log into Bugzilla

1. User ID
2. Password
3. And then click on **“Log in”**



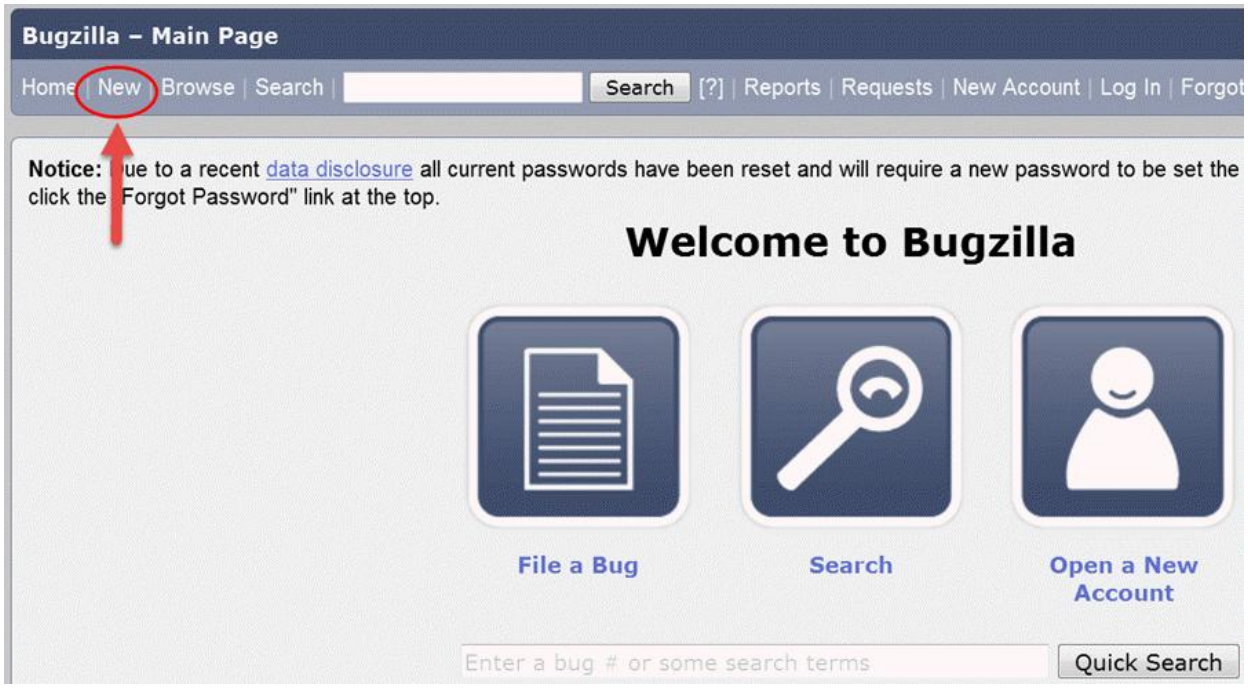
Step 3) You are successfully logged into Bugzilla system



With an account, the most common task is filing a new defect.

Creating a Bug-report in Bugzilla

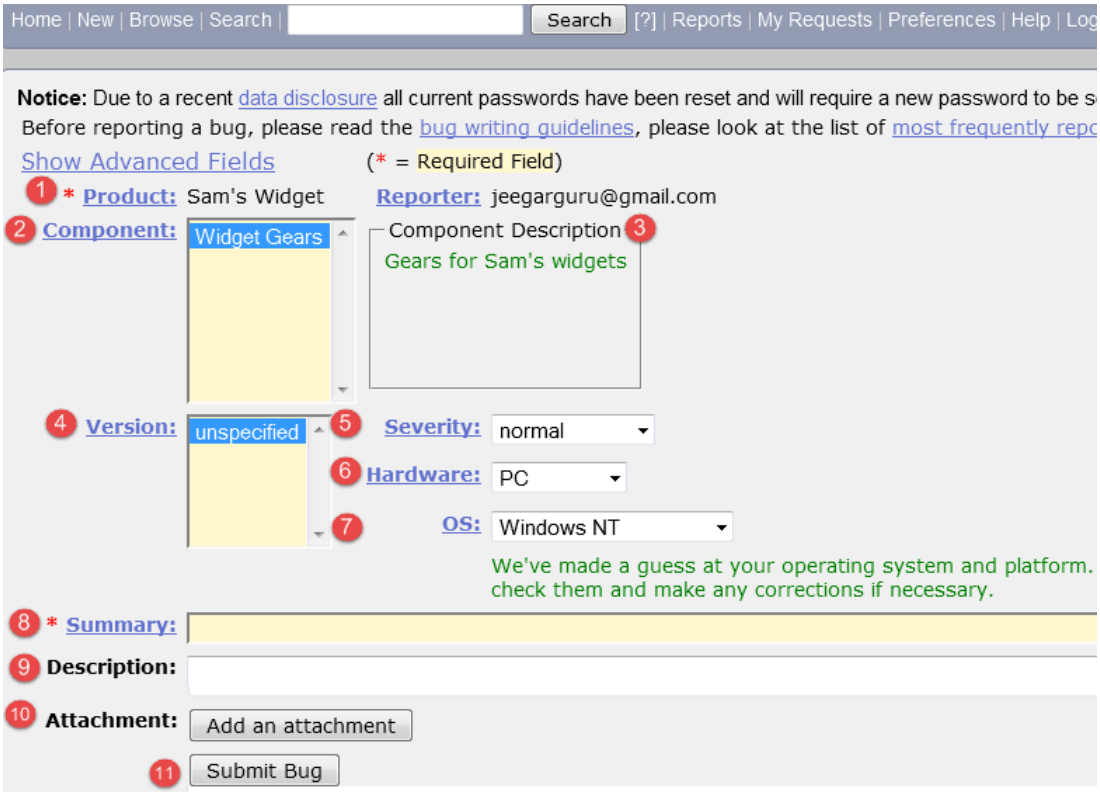
Step 1) To create a new bug in Bugzilla, visit the home-page of Bugzilla and click on **NEW** tab from the main menu



Step 2) In the next window

1. Enter Product
2. Enter Component
3. Give Component description
4. Select version,
5. Select severity
6. Select Hardware
7. Select OS
8. Enter Summary
9. Enter Description
10. Attach Attachment
11. Submit

NOTE: The above fields will vary as per your customization of Bugzilla



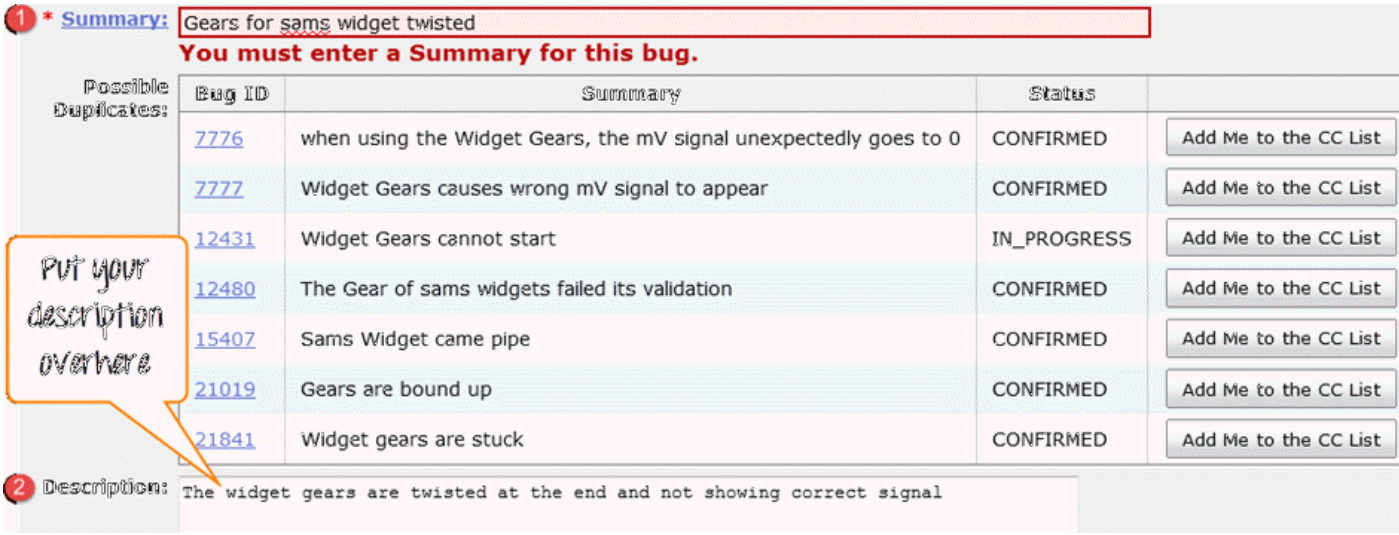
NOTE: The mandatory fields are marked with *.

In our case field's

- Summary
- Description

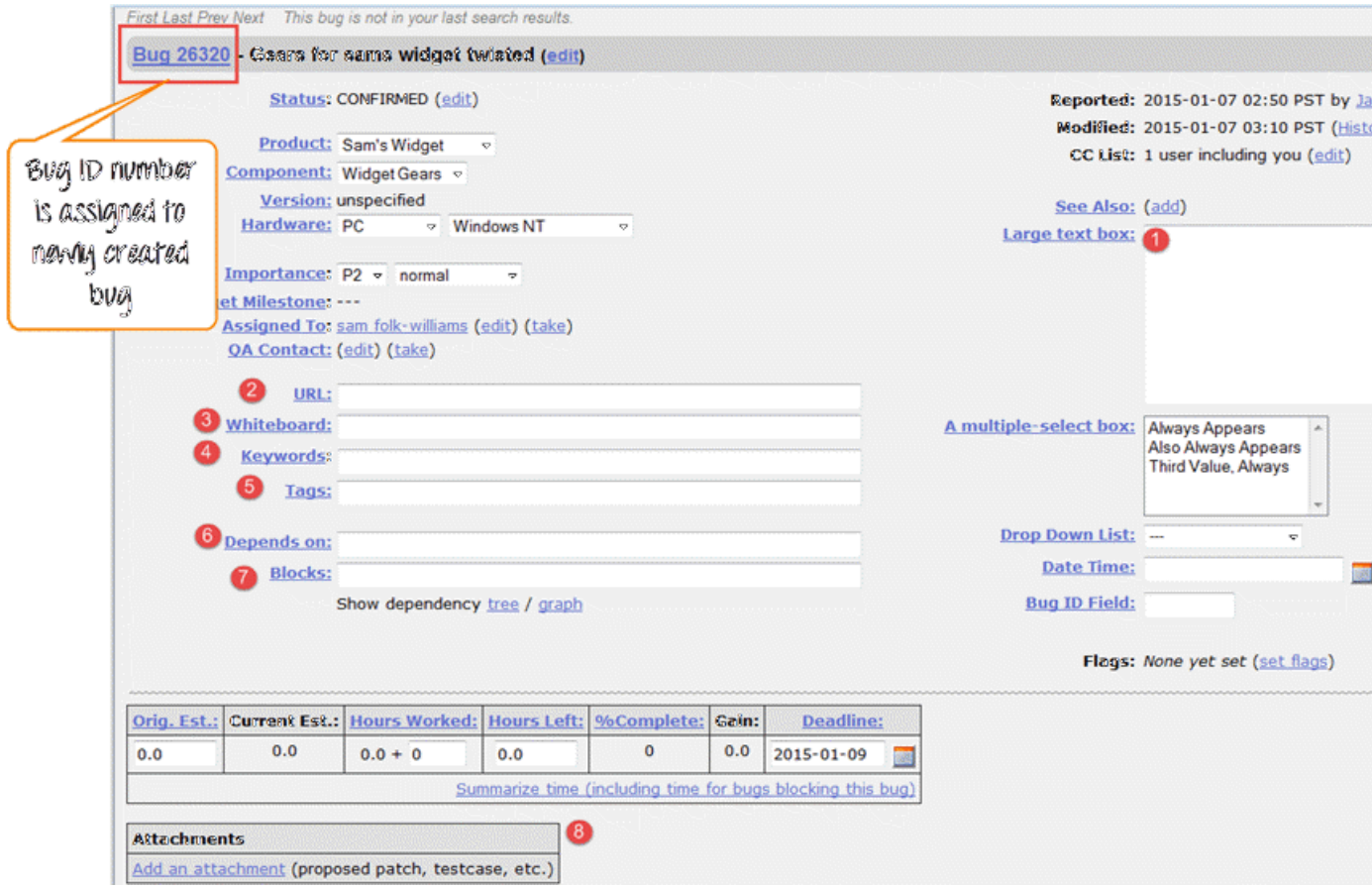
Are mandatory

If you do not fill them you will get a screen like below

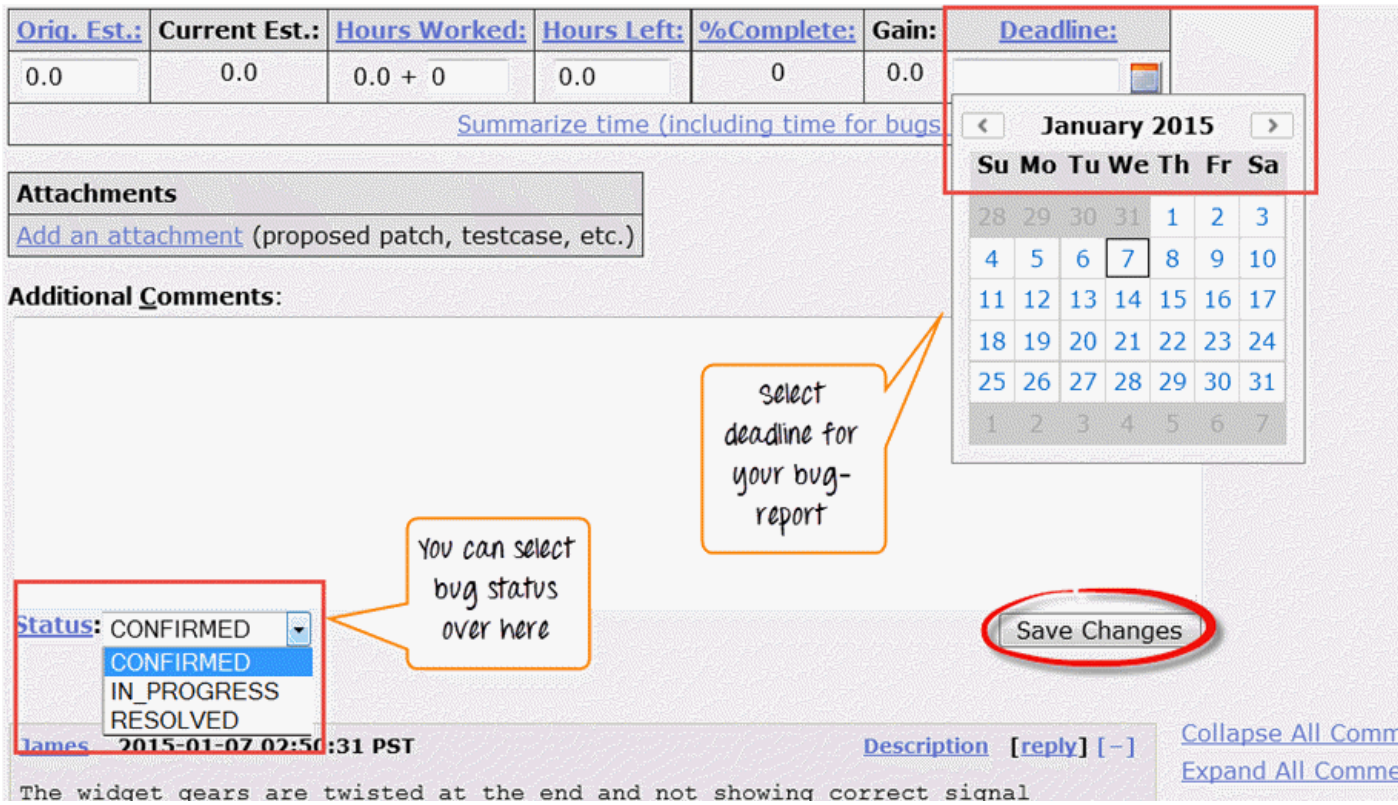


Step 4) Bug is created ID# 26320 is assigned to our Bug. You can also add additional information to the assigned bug like URL, keywords, whiteboard, tags, etc. This extra-information is helpful to give more detail about the Bug you have created.

1. Large text box
2. URL
3. Whiteboard
4. Keywords
5. Tags
6. Depends on
7. Blocks
8. Attachments



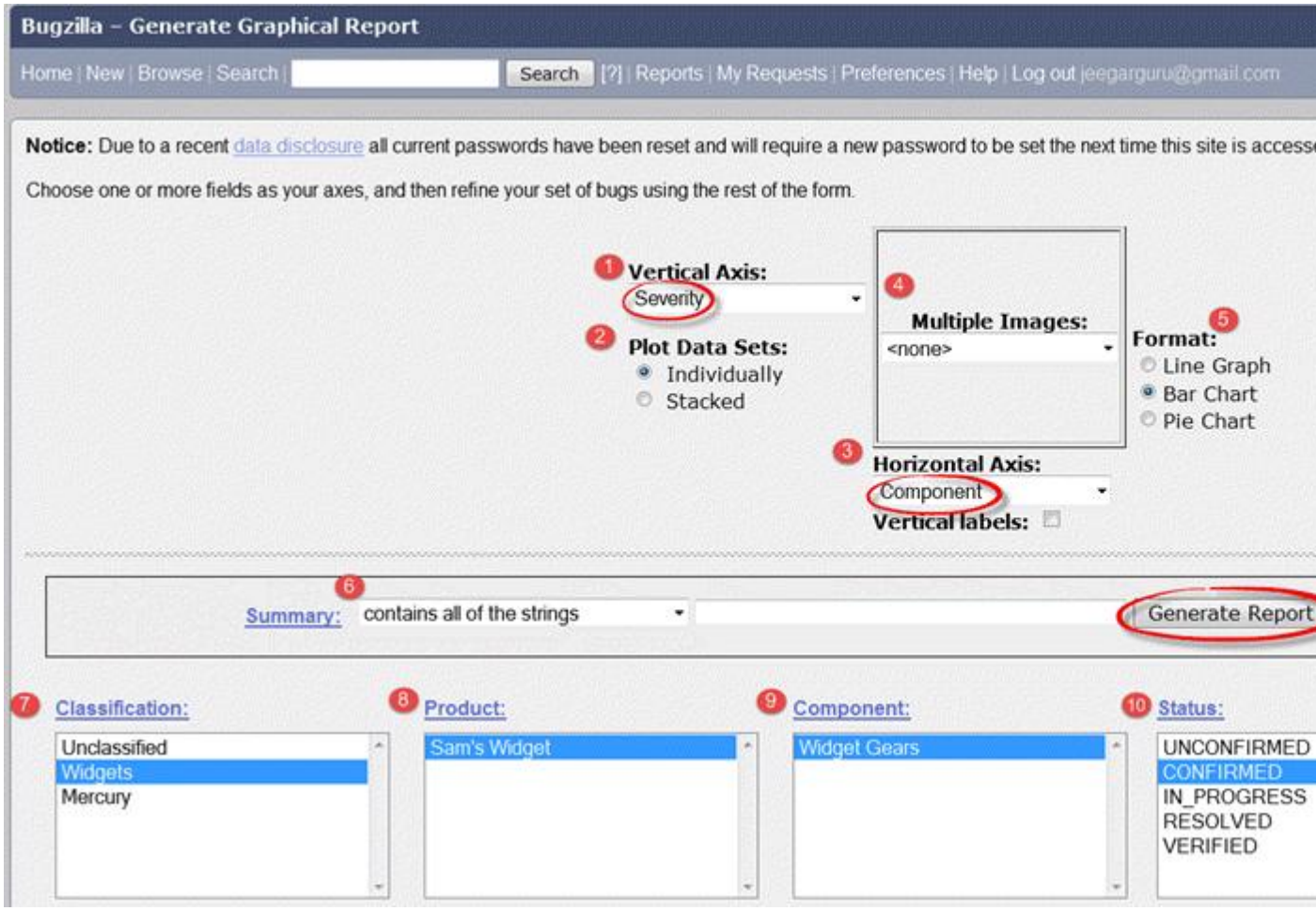
Step 5) In the same window if you scroll down further. You can select deadline date and also status of the bug. **Deadline in Bugzilla usually gives the time-limit to resolve the bug in given time frame.**



Create Graphical Reports

Graphical reports are one way to view the current state of the bug database. You can run reports either through an HTML table or graphical line/pie/bar-chart-based one. The idea behind graphical report in Bugzilla is to define a set of bugs using the standard search interface and then choosing some aspect of that set to plot on the horizontal and vertical axes. You can also get a 3-dimensional report by choosing the option of “Multiple Pages”.

Reports are helpful in many ways, for instance, if you want to know which component has the largest number of bad bugs reported against it. In order to represent that in the graph, you can select severity on X-axis and component on Y-axis, and then click on generate a report. It will generate a report with crucial information.


Bugzilla – Generate Graphical Report
Home | New | Browse | Search | Search [?] | Reports | My Requests | Preferences | Help | Log out jeegarguru@gmail.com
Notice: Due to a recent data disclosure all current passwords have been reset and will require a new password to be set the next time this site is access
Choose one or more fields as your axes, and then refine your set of bugs using the rest of the form.
1 Vertical Axis:
Severity
2 Plot Data Sets:
Individually
Stacked
4 Multiple Images:
<none>
5 Format:
Line Graph
Bar Chart
Pie Chart
3 Horizontal Axis:
Component
Vertical labels:
6 Summary:
contains all of the strings
Generate Report
7 Classification:
Unclassified
Widgets
Mercury
8 Product:
Sam's Widget
9 Component:
Widget Gears
10 Status:
UNCONFIRMED
CONFIRMED
IN_PROGRESS
RESOLVED
VERIFIED

Automate the process of identifying a list or dropdown box on a web page and determine the total number of items. Validate default selected value and perform selection testing.

Step 1: Create a Maven project

In Eclipse or STS:

1. Click File → New → Maven Project.
2. Enter the project name `DropdownTesting`.
3. Open `pom.xml` and add the Selenium dependency below.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.selenium</groupId>
    <artifactId>DropdownTesting</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
```

```
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.35.0</version>
    </dependency>
  </dependencies>

</project>
```

Save the file. If required, right-click the project and select Maven → Update Project.

Step 3: Create the Java class

1. Expand `src/main/java`.
2. Create a package named `com.selenium`.
3. Create a Java class named `DropdownSelectionTest`.
4. Copy the complete program below.

Step 4: Complete Java program

This program uses the DemoQA practice form, identifies its state dropdown, counts the available options, validates the default selection, and selects another option.

```
package com.selenium;

import java.time.Duration;
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.support.ui.Select;

public class DropdownSelectionTest {

    public static void main(String[] args) {

        WebDriver driver = null;

        try {
            // Launch Chrome
            driver = new ChromeDriver();
            driver.manage().window().maximize();

            driver.manage().timeouts()
                  .implicitlyWait(Duration.ofSeconds(5));

            // Open practice form
            driver.get("https://demoqa.com/automation-practice-form");

            Thread.sleep(3000);

            // Identify the State dropdown
            WebElement dropdownElement =
                    driver.findElement(By.id("state"));

            System.out.println("Dropdown identified successfully.");
```

```
// Create Select object
Select dropdown = new Select(dropdownElement);

// Count dropdown items
List<WebElement> options = dropdown.getOptions();

System.out.println("Total dropdown items: "
        + options.size());

// Display all dropdown options
System.out.println("\nAvailable dropdown items:");

for (WebElement option : options) {
   System.out.println(option.getText());
}

// Validate default selected value
String defaultValue =
      dropdown.getFirstSelectedOption().getText();

System.out.println("\nDefault selected value: "
        + defaultValue);

if (defaultValue.equals("Select State")) {
   System.out.println("Default value validation: PASS");
} else {
   System.out.println("Default value validation: FAIL");
```

```
    }

    // Perform selection testing
    dropdown.selectByVisibleText("NCR");

    Thread.sleep(1500);

    // Validate selected value
    String selectedValue =
          dropdown.getFirstSelectedOption().getText();

    System.out.println("\nSelected value: " + selectedValue);

    if (selectedValue.equals("NCR")) {
       System.out.println("Selection testing: PASS");
    } else {
       System.out.println("Selection testing: FAIL");
    }

} catch (Exception e) {
    System.out.println("Test failed: " + e.getMessage());
    e.printStackTrace();

} finally {
    if (driver != null) {
       try {
          Thread.sleep(3000);
```

```
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

            driver.quit();
            System.out.println("\nBrowser closed.");
        }
    }
}
```

Step 5: Execute the program

1. Save the Java file using `Ctrl + S`.
2. Right-click `DropdownSelectionTest.java`.
3. Select Run As → Java Application.
4. Chrome opens the practice form.
5. The program identifies the State dropdown, prints its options and count, checks the default value, and selects `NCR`.
6. View the results in the Eclipse/STS Console.

Output

Method	Purpose
`findElement(By.id("state"))`	Identifies the dropdown
`new Select(element)`	Creates a dropdown handler
`getOptions()`	Retrieves all dropdown options
`getFirstSelectedOption()`	Retrieves the currently selected option
`selectByVisibleText("NCR")`	Selects an option by its displayed text
`getText()`	Reads the displayed text